Playlist Manager API

Project & Database Documentation

ASP.NET Core .NET 10 | EF Core 10 | SQL Server | xUnit

Technical Reference Document
Generated as part of a full architecture & code audit

1. Project Overview

The Playlist Manager API is a RESTful backend for managing music playlists. Each user owns a set of playlists, and each playlist contains an ordered list of songs drawn from a shared song catalogue. The service is built on **ASP.NET Core (.NET 10)** with **Entity Framework Core 10** over **SQL Server**.

The application is layered so that each layer has one responsibility:

Layer	Responsibility
Controllers	HTTP routing, model validation, status codes.
Services	Business rules: ownership, duplicate handling, ordering, DTO mapping.
AppDbContext	EF Core mapping and relationships; serves as Repository + Unit of Work.
Models	Database entities (User, Playlist, Song, PlaylistSongs).
DTOs	Request/response contracts with validation attributes.
Middleware	Global exception handler emitting RFC 7807 ProblemDetails.

Core Features

- Create users and a shared song catalogue.
- Create a playlist owned by a specific user.
- Add a song to one of your own playlists, with ownership, existence, ordering, and duplicate checks.
- Retrieve all playlists belonging to a given user.
- Update and delete playlists (owner-checked; delete cascades to song links).

Technology Stack

- ASP.NET Core Web API on .NET 10
- Entity Framework Core 10 (SQL Server provider)
- Scalar for interactive API documentation (development only)
- xUnit v3, FluentAssertions, FakeItEasy, EF Core InMemory for testing

2. API Reference

Base path: `/api`. All responses are JSON. Errors use the ProblemDetails shape.

Method	Route	Description	Success
POST	/api/user	Create a user	201
GET	/api/user	List all users	200 / 204
GET	/api/user/search/id/{id}	Get user by id	200 / 404
GET	/api/user/search/name/{name}	Search users by name	200 / 404
POST	/api/playlist	Create a playlist	201 / 400
POST	/api/playlist/addsong	Add a song to a playlist	201 / 400 / 409
GET	/api/playlist/search/userId/{userId}	All playlists for a user	200
GET	/api/playlist/search/playlistId/{id}	Single playlist by id	200 / 404
GET	/api/playlist/search/name/{name}	Search playlists by name	200 / 404
PUT	/api/playlist/update	Rename a playlist (owner-checked)	200 / 404
DELETE	/api/playlist?playlistId={id}&userId={id}	Delete a playlist (cascades)	204 / 404
GET	/api/song	List all songs	200 / 204
POST	/api/song	Create a song	201
GET	/api/song/search/id/{id}	Get song by id	200 / 404
GET	/api/song/search/name/{name}	Search songs by name	200 / 404
GET	/api/song/search/artist/{artist}	Search songs by artist	200 / 404

Example requests

```
POST /api/playlist
{ "name": "Road Trip", "userId": 1 }

POST /api/playlist/addsong
{ "playlistId": 1, "songId": 1, "userId": 1 }
```

3. Setup & Operations

Prerequisites

- .NET 10 SDK (the runtime alone is insufficient for migrations).
- SQL Server Express (default) or another supported provider.
- Visual Studio 2022/2026, or any editor plus the `dotnet ef` CLI.

Configuration

The connection string is in `appsettings.json` under `ConnectionStrings:DefaultConnection`:

```
Server=localhost\SQLExpress;Database=StorageDb;Trusted_Connection=True;TrustServerCertificate=True
```

Run, Migrate, Test

```
# apply the shipped migration
dotnet ef database update

# run the API (Scalar docs at https://localhost:7230/scalar in Development)
dotnet run --project PlaylistManagerApi

# run the test suite
dotnet test
```

> **Seeding:** creating a playlist requires an existing user. Use `POST /api/user` or the bundled `data_insertions.txt` script (inserts in dependency order: Users & Songs, then Playlists, then PlaylistSongs).

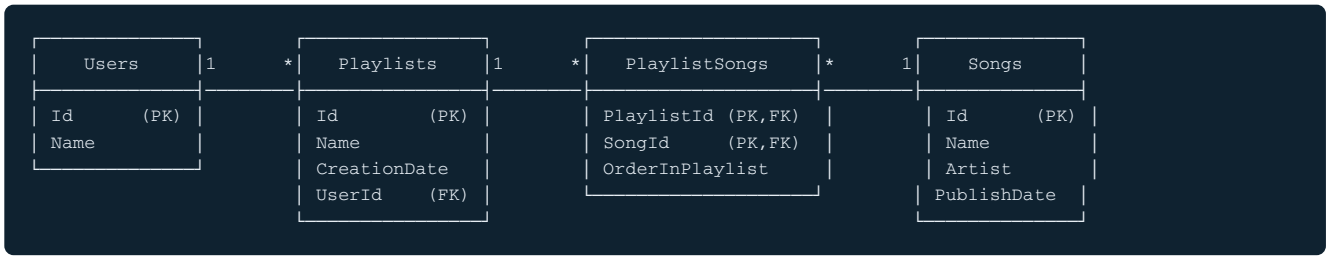
Database Documentation

4. Overview

The database is a normalized relational schema (SQL Server) consisting of four tables: `Users`, `Songs`, `Playlists`, and the join table `PlaylistSongs`. It models two relationships: a one-to-many between users and playlists, and a many-to-many between playlists and songs carried by the join table (which additionally stores an explicit ordering position).

5. ERD Description

The entity-relationship structure is summarised below. **PK** marks primary-key columns; **FK** marks foreign keys.



- **Users → Playlists**: one-to-many. A user owns zero or more playlists; every playlist has exactly one owner.
- **Playlists ↔ Songs**: many-to-many, resolved by `PlaylistSongs`. A playlist contains many songs; a song may belong to many playlists.
- **OrderInPlaylist**: payload on the join, giving each song a position within its playlist.

6. Table & Column Descriptions

Users

Column	Type	Null	Key	Description
Id	int (IDENTITY)	No	**PK**	Surrogate identifier, auto-incremented.
Name	nvarchar(200)	No		Display name. Required, max length 200.

Songs

Column	Type	Null	Key	Description
Id	int (IDENTITY)	No	**PK**	Surrogate identifier, auto-incremented.
Name	nvarchar(200)	No		Song title. Required, max length 200.
Artist	nvarchar(200)	No		Performing artist. Required, max length 200.
PublishDate	datetime2	No		Stored in UTC. Currently set at creation time (see business rules).

Playlists

Column	Type	Null	Key	Description
Id	int (IDENTITY)	No	**PK**	Surrogate identifier, auto-incremented.
Name	nvarchar(200)	No		Playlist name. Required, max length 200.
CreationDate	datetime2	No		UTC timestamp set when the playlist is created.
UserId	int	No	**FK**	Owner; references `Users.Id`. Indexed.

PlaylistSongs (join)

Column	Type	Null	Key	Description
PlaylistId	int	No	**PK** **FK**	References `Playlists.Id`. Part of composite key.
SongId	int	No	**PK** **FK**	References `Songs.Id`. Part of composite key. Indexed.
OrderInPlaylist	int	No		1-based position of the song within the playlist.

7. Keys, Constraints & Indexes

Primary keys

Table	Primary key	Notes
Users	Id	Single-column surrogate, IDENTITY(1,1).
Songs	Id	Single-column surrogate, IDENTITY(1,1).
Playlists	Id	Single-column surrogate, IDENTITY(1,1).
PlaylistSongs	(PlaylistId, SongId)	Composite. Guarantees a song appears at most once per playlist.

Foreign keys

Constraint	Column	References	On delete
FK_Playlists_Users_UserId	Playlists.UserId	Users.Id	CASCADE
FK_PlaylistSongs_Playlists_PlaylistId	PlaylistSongs.PlaylistId	Playlists.Id	CASCADE
FK_PlaylistSongs_Songs_SongId	PlaylistSongs.SongId	Songs.Id	CASCADE

Indexes

Index	Table	Column(s)	Purpose
PK_PlaylistSongs	PlaylistSongs	(PlaylistId, SongId)	Composite PK; also covers lookups by playlist.
IX_Playlists_UserId	Playlists	UserId	Efficient "all playlists for a user" queries.
IX_PlaylistSongs_SongId	PlaylistSongs	SongId	Reverse lookups (which playlists contain a song) and FK support.

> SQL Server permits the two cascading foreign keys on `PlaylistSongs` because they originate from two independent roots (Playlists and Songs) and do not form a single multi-path cascade or cycle.

8. Referential Integrity & Cascade Behaviour

- Deleting a **User** cascades to their **Playlists**, which in turn cascade to their **PlaylistSongs**.
- Deleting a **Playlist** cascades to its **PlaylistSongs** rows; the songs themselves are untouched.
- Deleting a **Song** cascades to its **PlaylistSongs** rows; playlists remain, simply without that song.
- A playlist cannot reference a non-existent user, and a join row cannot reference a non-existent playlist or song.

9. Business Rules

- A playlist must belong to an existing user (enforced in the service and by the FK).
- A song can be added to a playlist only by that playlist's owner (ownership check on `UserId`).
- The same song cannot appear twice in one playlist (composite PK + service-level pre-check).
- `OrderInPlaylist` is assigned as `MAX(existing) + 1`, starting at 1.
- Duplicate playlist *names* are permitted; only duplicate *songs within a playlist* are blocked.
- `CreationDate` and `PublishDate` are stored in UTC. `PublishDate` currently captures creation time because the create-song contract does not yet accept a real release date.

10. Normalization Notes

The schema is in **Third Normal Form (3NF)**:

- **1NF** — all columns hold atomic values; there are no repeating groups (song membership is rows in the join table, not

a list column).

- **2NF** — in the composite-key join table, the only non-key attribute (`OrderInPlaylist`) depends on the whole key, not part of it.
- **3NF** — no transitive dependencies; song attributes live only in `Songs`, user attributes only in `Users`, and are referenced by id rather than duplicated.

11. Assumptions

- Ownership is currently asserted via a client-supplied `UserId`; a production system must derive it from an authenticated identity.
- Songs form a shared catalogue, so identical title/artist pairs may legitimately coexist (no uniqueness constraint on songs).
- User names are not unique by design; search returns all matches.

12. Migrations

The schema is produced by a single EF Core migration, `Initial` (`20260626113305_Initial`). It creates the four tables, the three foreign keys with cascade delete, the composite primary key on `PlaylistSongs`, and the two supporting indexes. The migration matches the model snapshot, so `dotnet ef database update` generates exactly the schema described above. Apply with:

```
dotnet ef database update      # CLI
Update-Database                # Package Manager Console
```

When the model changes, add a new, differently named migration (for example `Add-Migration AddPlaylistDescription`) rather than recreating `Initial`.